

CP Duel Backend Interview Guide

Complete backend explanation with file-by-file, function-by-function notes and project flow summary.

Scope: backend only. Frontend implementation details are intentionally omitted except where needed to explain backend flow.

Complete Project Flow

Step	Backend responsibility
1. Register or log in	User submits credentials and Codeforces handle. Backend creates/verifies the user and returns a JWT.
2. Create or join a room	Authenticated REST APIs create a MongoDB Room or add a second player to it.
3. Join Socket.IO room	Frontend emits createRoom/joinRoom so both clients subscribe to the same real-time channel.
4. Host starts duel	Protected REST endpoint validates host, player count, and room state.
5. Select Codeforces problem	Backend fetches problems and solved submissions, filters by rating and eligibility, then picks one.
6. Persist duel state	Room is updated with currentProblem, status ongoing, and startedAt.
7. Notify players	Socket.IO emits duelStarted with problem details and start time.
8. Poll submissions	Server checks Codeforces user.status every 6 seconds for both handles.
9. Determine winner	First valid Accepted submission after startedAt wins.
10. Finish duel	Room and User documents are updated, ratings change, and gameResult is emitted.

Interview summary: REST APIs are the source of truth for durable state changes; Socket.IO is used to synchronize live UI state; Codeforces is the external judge used for problem selection and winner verification.

Backend Architecture

The backend is a Node.js and Express application backed by MongoDB through Mongoose. It exposes protected REST routes for authentication, rooms, and duels. A Socket.IO server sits on the same HTTP server so live lobby and result events can be broadcast to both players.

Layer	Files	Responsibility
Entry point	src/server.js	Loads env vars, connects MongoDB, configures Express/CORS, mounts routes, initializes Socket.IO.
Config	src/config/db.js	Creates the MongoDB connection using MONGO_URI.
Models	src/models/User.js, src/models/Room.js	Define persisted users, rooms, duel state, problem data, and match results.
Middleware	src/middleware/auth.js	Verifies Bearer JWT and attaches req.user.
Routes	src/routes/*.js	Map URL endpoints to controller functions.
Controllers	src/controllers/*.js	Implement request/response logic and coordinate services/models.
Services	src/services/*.js	External Codeforces calls, problem selection, polling, and rating updates.
Sockets	src/sockets/index.js	Socket.IO room membership and real-time event broadcasting.

Key design decision

The backend keeps real state in MongoDB, not in sockets. Socket.IO only tells connected clients what changed. This means room refreshes can recover from the database, and the client cannot simply emit a winner event to cheat.

Data Models

User model - src/models/User.js

Field or method	Explanation
username	Unique display/login identity, trimmed, 3 to 20 characters.
email	Unique lowercased email address.
password	Stored as bcrypt hash, never returned from JSON responses.
codeforcesHandle	Handle used for fetching submissions from Codeforces.
rating	Internal duel rating, default 1200.
wins / losses	Simple match statistics updated after duel completion.
pre('save') hook	Hashes password only when password is modified.
comparePassword()	Compares a plaintext candidate password with the stored hash.
toJSON()	Removes password before sending user objects in API responses.

Room model - src/models/Room.js

Field	Explanation
roomId	Public room code generated with nanoid(8).toUpperCase().
host	User ObjectId of the room creator. Only host can start duel.
players	Embedded player snapshots: user id, username, Codeforces handle, and rating.
status	Lifecycle: waiting, ready, ongoing, finished.

Field	Explanation
currentProblem	Selected Codeforces problem with contestId, index, name, rating, url, and tags.
winner / winnerUsername	Winner user id and denormalized username for display.
winnerSubmissionTime	Timestamp of Accepted submission used to win.
ratingDelta / winnerNewRating / loserNewRating	Stores match rating outcome for history/result views.
startedAt / finishedAt	Duel timing used for filtering stale submissions and result display.

API And Socket Contract

REST API

Method	Endpoint	Auth	Controller	Purpose
POST	/api/auth/register	No	register	Create user and return JWT.
POST	/api/auth/login	No	login	Verify credentials and return JWT.
GET	/api/auth/me	Yes	getMe	Return authenticated user.
POST	/api/rooms/create	Yes	createRoom	Create waiting room with current user as host.
POST	/api/rooms/join	Yes	joinRoom	Join waiting room and mark it ready.
GET	/api/rooms/history	Yes	getHistory	Return recent finished matches for current user.
GET	/api/rooms/:id	Yes	getRoom	Fetch room by public roomId.
POST	/api/duel/start	Yes	startDuel	Host starts duel; selects problem; starts polling.
GET	/api/duel/status/:roomId	Yes	getDuelStatus	Return current duel state.

Socket.IO events

Direction	Event	Payload	Purpose
Client to server	createRoom	{ roomId, username }	Host joins the socket room after REST room creation.
Client to server	joinRoom	{ roomId, username, players }	Opponent joins socket room; server broadcasts playerJoined.
Client to server	startGame	{ roomId }	Informational broadcast that start was requested.
Client to server	leaveRoom	{ roomId, username }	Socket leaves room and broadcasts playerLeft.
Server to client	playerJoined	{ players, roomId }	Lobby updates when second player joins.
Server to client	gameStarting	{ roomId }	Host clicked start.
Server to client	duelStarted	{ problem, startedAt }	Backend selected problem and duel is live.
Server to client	gameResult	{ winner, submissionTime, problem, ratingDelta, ... }	Winner verified and match finished.
Server to client	playerLeft	{ username }	Player left room.

File-By-File And Function-By-Function Explanation

Use this section as the main interview preparation reference. Each file is grouped by responsibility, then each function or important constant is explained in plain backend terms.

src/server.js

Item	Explanation
Purpose	Application entry point. Creates Express app and HTTP server, connects MongoDB, configures CORS/body parsing, registers route modules, initializes Socket.IO, and starts listening.
connectDB()	Called once during startup to establish the MongoDB connection before serving requests.
allowedOrigins	Allows CLIENT_URL plus local Vite/React ports. Requests without origin are allowed for tools like curl.
app.get('/health')	Simple health check returning { status: 'ok' }. Useful for deployment platforms.
initSocket(server)	Attaches Socket.IO to the same HTTP server so REST and sockets share one backend process.

src/config/db.js

Item	Explanation
connectDB()	Async MongoDB connector. Uses mongoose.connect(process.env.MONGO_URI), logs the connected host, and exits the process if connection fails.

src/middleware/auth.js

Item	Explanation
authMiddleware()	Reads Authorization header, expects Bearer token, verifies JWT with JWT_SECRET, fetches the user without password, attaches it as req.user, then calls next().
Failure paths	Returns 401 for missing token, invalid/expired token, or user id that no longer exists.

src/routes/auth.js

Item	Explanation
router.post('/register')	Maps registration to register().
router.post('/login')	Maps login to login().
router.get('/me')	Protected route that returns current user through getMe().

src/routes/rooms.js

Item	Explanation
router.post('/create')	Protected endpoint to create a new room.
router.post('/join')	Protected endpoint to join an existing waiting room.
router.get('/history')	Protected history endpoint. It must appear before /:id so 'history' is not treated as a room id.
router.get('/:id')	Protected endpoint to fetch one room by roomId.

src/routes/duel.js

Item	Explanation
router.post('/start')	Protected endpoint that starts the duel.
router.get('/status/:roomId')	Protected endpoint that returns current duel status.

src/controllers/authController.js

Item	Explanation
generateToken(userId)	Signs a JWT containing userId. Token expires in 7 days.
register(req, res)	Validates required fields, prevents duplicate username/email, creates the user, and returns token plus user.
login(req, res)	Finds user by email, validates password with comparePassword(), then returns token plus user.
getMe(req, res)	Returns req.user set by authMiddleware.

src/controllers/roomController.js

Item	Explanation
createRoom(req, res)	Creates a Room with current user as host and first player. Initial status is waiting.
joinRoom(req, res)	Validates roomId, room existence, waiting status, capacity, and duplicate joins. Adds current user and marks room ready.
getRoom(req, res)	Fetches a room by public roomId and returns it.
getHistory(req, res)	Finds finished rooms involving current user, derives opponent/outcome/rating information, and returns a compact match history.

src/controllers/duelController.js

Item	Explanation
setIO(io)	Stores Socket.IO server reference so REST controller can emit live events.
startDuel(req, res)	Validates host and room readiness, calculates average rating, selects Codeforces problem, stores room state, emits duelStarted, and starts polling.
getDuelStatus(req, res)	Returns status, currentProblem, winner, startedAt, and finishedAt for a room.

src/services/codeforcesService.js

Item	Explanation
fetchProblems()	Calls Codeforces problemset.problems and returns all problems if API status is OK.
fetchSolvedProblems(handle)	Calls Codeforces user.status for recent submissions, collects accepted problem keys into a Set.
selectProblem(handle1, handle2, avgRating)	Fetches problems and solved sets in parallel, filters by rating range, unsolved status, non-Gym contest, valid index, and banned tags, then randomly picks a candidate.
getRecentSubmissions(handle, count)	Fetches recent submissions for polling. Returns an empty array if Codeforces response is not OK.

src/services/pollingService.js

Item	Explanation
activePolls	In-memory Map from roomId to interval id. Prevents duplicate polling loops for the same room.
startPolling(roomId, problem, players, io, startedAt)	Every 6 seconds, fetches recent submissions for each player and searches for an Accepted submission for the exact selected problem after startedAt.
stopPolling(roomId)	Clears and removes an active interval for a room.
calcEloDelta(winnerRating, loserRating)	Computes Elo-style rating change with $K = 32$.
handleWinner(roomId, winner, io)	Loads ongoing room, identifies winner/loser, updates room and users, and emits gameResult.

src/sockets/index.js

Item	Explanation
socketRoomMap	Tracks socket.id to roomId for cleanup on disconnect.
initSocket(server)	Creates Socket.IO server with allowed origins and websocket/polling transports, passes io into duelController, and registers connection handlers.
createRoom event	Socket joins the room channel after REST room creation.
joinRoom event	Socket joins room and emits playerJoined to everyone in that room.
startGame event	Broadcasts gameStarting. Actual duel start still happens through REST /api/duel/start.
leaveRoom event	Socket leaves room and broadcasts playerLeft.
disconnect event	Cleans socketRoomMap entry for disconnected socket.

Deep Flow Walkthroughs

Authentication flow

- Client sends register/login request.
- Controller validates input and checks user in MongoDB.
- User model hashes password before save during registration.
- Login compares candidate password with bcrypt hash.
- Controller signs JWT containing userId and returns it.
- Protected requests include Authorization: Bearer <token>.
- authMiddleware verifies JWT, fetches user, and attaches req.user.

Room and lobby flow

- Authenticated host calls POST /api/rooms/create.
- Room is saved with host as first player and status waiting.
- Host socket emits createRoom and joins Socket.IO room.
- Opponent calls POST /api/rooms/join using roomId.
- Backend validates room is waiting and has fewer than two players.
- Opponent is added, room status becomes ready.
- Opponent socket emits joinRoom; server broadcasts playerJoined.

Duel start and winner flow

- Host calls POST /api/duel/start.
- Backend verifies requester is the room host.
- Average rating is calculated from room.players.
- selectProblem fetches Codeforces data and chooses an eligible unsolved problem.
- Room is updated to ongoing with currentProblem and startedAt.
- Socket.IO emits duelStarted to the room.
- startPolling checks recent Codeforces submissions every 6 seconds.
- Only Accepted submissions after startedAt for the exact problem count.
- handleWinner stops polling, updates room, updates ratings/stats, and emits gameResult.

Interview-Ready Talking Points

One-minute backend pitch

CP Duel's backend is a Node.js Express service with MongoDB persistence and Socket.IO real-time updates. Users authenticate using JWT. A Room document represents the complete duel lifecycle: lobby, players, selected Codeforces problem, status, winner, and rating outcome. The host starts a duel through a protected REST endpoint; the backend selects a fair Codeforces problem by rating and solved history, then polls Codeforces submissions server-side. When it verifies the first Accepted solution after the duel start time, it updates MongoDB and broadcasts the result to both players.

Strong points to mention

- Server-side winner verification prevents clients from self-declaring a win.
- MongoDB is the durable source of truth; Socket.IO is only for real-time synchronization.
- Problem selection uses both rating and solved history to keep matches fair.

- JWT middleware centralizes authentication for protected APIs.
- Room status creates a clear state machine: waiting -> ready -> ongoing -> finished.
- Polling intervals are deduplicated with activePolls so the same room does not poll twice.

Weaknesses and improvement answers

Question	Good answer
What happens if the server restarts mid-duel?	Current in-memory polling would stop. A production version should use persistent jobs, a queue, or restart recovery that resumes ongoing rooms.
How would you handle Codeforces rate limits?	Cache problemset data, add retry/backoff, reduce polling frequency, and gracefully show delayed verification.
Can database updates become inconsistent?	Yes, winner room update and user rating updates are separate. Use MongoDB transactions for stronger consistency.
How do you handle simultaneous solves?	Current logic finds the first non-null result from player order. Better logic should compare submission timestamps and choose earliest valid accepted submission.
Why not use only sockets?	REST gives authenticated, durable state transitions. Sockets are best for broadcasting changes, not replacing the data model.

Quick Reference

Environment variables

Variable	Purpose
PORT	Backend port, default 5000.
MONGO_URI	MongoDB connection string.
JWT_SECRET	Secret used to sign and verify JWTs.
CLIENT_URL	Frontend origin allowed by CORS and Socket.IO.

Dependencies

Package	Used for
express	HTTP API routing and middleware.
mongoose	MongoDB schemas, models, queries, and updates.
jsonwebtoken	JWT creation and verification.
bcryptjs	Password hashing and comparison.
socket.io	Real-time room/lobby/result events.
axios	Codeforces API calls.
cors	Allowed frontend origin configuration.
dotenv	Environment variable loading.
nanoid	Human-shareable room id generation.

Core backend sentence

The backend coordinates a two-player Codeforces race by persisting room state, selecting a fair problem, verifying submissions server-side, updating ratings, and broadcasting live state changes.